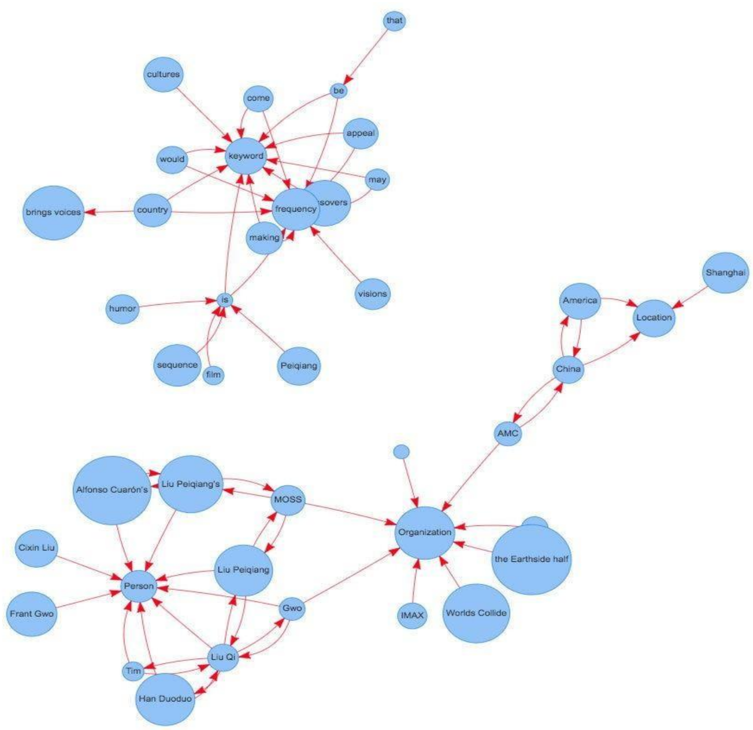
Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** Representation of a document with their keywords using TextRank	
Experiment No: 09	**Date:**	**Enrolment No: 92301733023**

Aim: Representation of a document with their keywords using TextRank

IDE: Google Colab **Theory:**

TextRank is an algorithm based on PageRank, which often used in keyword extraction and text summarization. In this article, I will help you understand how TextRank works with a keyword extraction example and show the implementation by Python.

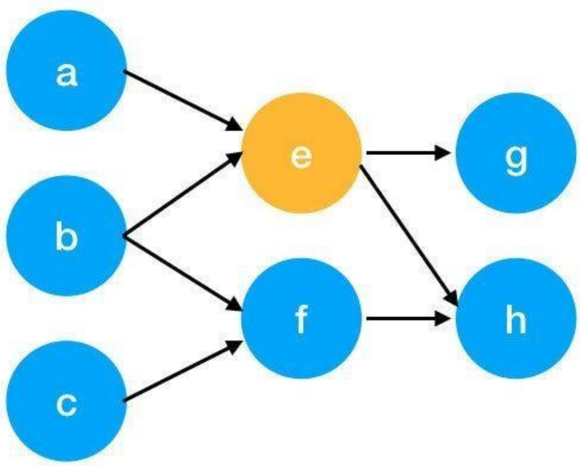


Understand PageRank

There are tons of articles talking about PageRank, so I just give a brief introduction to PageRank. This will help us understand TextRank later because it is based on PageRank. PageRank (PR) is an algorithm used to calculate the weight for web pages. We can take all web pages as a big directed graph. In this graph, a node is a webpage. If webpage A has the link to web page B, it can be represented as a directed edge from A to B. After we construct the whole graph, we can assign weights for web pages by the following formula.

$$S(V_i) = (1-d) + d * \sum_{j \in In(v_i)} \frac{1}{|Out(V_j)|} S(V_j)$$

- S(Vi) - the weight of webpage i
- d - damping factor, in case of no outgoing links
- In(Vi) - inbound links of i, which is a set
- Out(Vj) - outgoing links of j, which is a set
- |Out(Vj)| - the number of outbound links



Here is an example to better understand the notation above. We have a graph to represent how web pages link to each other. Each node represents a webpage, and the arrows represent edges. We want to get the weight of webpage e. We can rewrite the summation part in the above function to a simpler version.

$$In(v_e) = \{a, b\}, j \in \{a, b\}$$

$$\begin{aligned}\sum_{j \in \{a,b\}} \frac{1}{|Out(V_j)|} S(V_j) &= \frac{1}{|Out(V_a)|} S(V_a) + \frac{1}{|Out(V_b)|} S(V_b) \\ &= \frac{1}{|\{e\}|} S(V_a) + \frac{1}{|\{e, f\}|} S(V_b) \\ &= S(V_a) + \frac{1}{2} S(V_b)\end{aligned}$$

We can get the weight of webpage e by the following function.

$$S(V_e) = (1 - d) + d * \left(S(V_a) + \frac{1}{2} S(V_b) \right)$$

We can see the weight of the webpage e is dependent on the weights of its inbound pages. We need to run this iteration much time to get the final weight. In the initialization, the importance of each webpage is 1.

Pre Lab Exercise:

1. What are the key components of TextRank?

- **Graph Construction:** Words or sentences are represented as nodes, and edges are formed based on co-occurrence or similarity.
- **Text Preprocessing:** Includes tokenization, stop-word removal, and normalization.
- **Weight Assignment:** Edges are weighted based on frequency or similarity between words.
- **Ranking Algorithm:** Uses an iterative scoring method (similar to PageRank) to rank important words or sentences.
- **Convergence:** The algorithm runs until scores stabilize.

2. What are some limitations of TextRank for keyword extraction?

- Does not understand **semantic meaning**, only relies on word co-occurrence.
- Performance depends on **quality of preprocessing**.
- May extract **irrelevant or redundant keywords**.
- Not effective for **small datasets or short texts**.
- Cannot capture **deep contextual relationships** like modern NLP models.

3. What are the advantages of TextRank for keyword extraction?

- **Unsupervised method** (no training data required).
- Simple and easy to implement.
- Works well for **large text documents**.
- Language-independent approach.
- Can be used for both **keyword extraction and text summarization**.

Program (Code):

```
import nltk
import networkx as nx
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import string

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')

def textrank_keywords(text, window_size=4, top_n=5):

    words = word_tokenize(text.lower())

    stop_words = set(stopwords.words('english'))
    words = [w for w in words if w not in stop_words and w not in string.punctuation]

    graph = nx.Graph()

    for i in range(len(words)):
        for j in range(i + 1, min(i + window_size, len(words))):
            if words[i] != words[j]:
                graph.add_edge(words[i], words[j])

    scores = nx.pagerank(graph)

    ranked_words = sorted(scores.items(), key=lambda x: x[1], reverse=True)

    keywords = [word for word, score in ranked_words[:top_n]]
```

```
    return keywords

text = """
Machine learning is a field of artificial intelligence that uses statistical techniques
to give computer systems the ability to learn from data without being explicitly programmed.
"""

keywords = textrank_keywords(text)

print("Extracted Keywords:", keywords)
```

Results:

```
Extracted Keywords: ['artificial', 'data', 'intelligence', 'learn', 'uses']
```

Observation:

- The TextRank algorithm works by building a graph of words and applying the PageRank algorithm to rank important words.
- It does not require labeled data, making it an **unsupervised method**.
- Stopwords and punctuation removal significantly improve keyword quality.
- The window size affects how words are connected in the graph.
- The algorithm extracts meaningful keywords but sometimes includes **generic or less relevant words**.
- It focuses on **frequency + co-occurrence**, not actual meaning or context.

Post Lab Exercise:

I am a passionate and curious learner currently pursuing my studies in the field of Information and Communication Technology. I have a strong interest in software development, especially in building real-world applications that solve practical problems.

I enjoy working with technologies like Flutter, React, Node.js, and SQL. I have developed projects such as a Daily Expense Tracker, a Library Management System, and currently, I am working on an innovative project called **HiddenHive**, which helps users discover hidden tourist places across Gujarat using smart recommendations.

I like exploring new technologies and improving my problem-solving skills through coding and practical implementation. I am also interested in UI/UX design and always try to make my applications visually appealing and user-friendly.

Apart from technical skills, I believe in teamwork and collaboration. I enjoy working in a group where we can share ideas and build something meaningful together. I am always motivated to learn new things and improve myself continuously.
My goal is to become a skilled software developer and contribute to building impactful digital solutions.

Code :

```
import nltk
import networkx as nx
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import string

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')

def textrank_keywords(text, window_size=4, top_n=25):
```

```
    words = word_tokenize(text.lower())

    stop_words = set(stopwords.words('english'))
    words = [w for w in words if w not in stop_words and w not in string.punctuation]

    graph = nx.Graph()

    for i in range(len(words)):
        for j in range(i + 1, min(i + window_size, len(words))):
            if words[i] != words[j]:
                graph.add_edge(words[i], words[j])

    scores = nx.pagerank(graph)

    ranked_words = sorted(scores.items(), key=lambda x: x[1], reverse=True)

    keywords = [word for word, score in ranked_words[:top_n]]

    return keywords

text = """(I am a passionate and curious learner currently pursuing my studies in the field of Information and
Communication Technology. I have a strong interest in software development, especially in building real-world
applications that solve practical problems.

I enjoy working with technologies like Flutter, React, Node.js, and SQL. I have developed projects such as a
Daily Expense Tracker, a Library Management System, and currently, I am working on an innovative project
called HiddenHive, which helps users discover hidden tourist places across Gujarat using smart
recommendations.

I like exploring new technologies and improving my problem-solving skills through coding and practical
implementation. I am also interested in UI/UX design and always try to make my applications visually appealing
and user-friendly.

Apart from technical skills, I believe in teamwork and collaboration. I enjoy working in a group where we can
share ideas and build something meaningful together. I am always motivated to learn new things and improve
myself continuously.
My goal is to become a skilled software developer and contribute to building impactful digital solutions.
)"""
```

```
keywords = textrank_keywords(text)

print("Top 25 Keywords:", keywords)
```

Output :

```
Top 25 Keywords: ['working', 'currently', 'always', 'new', 'building', 'applications', 'skills', 'software', 'practical', 'like', 'enjoy', 'technologies', 'pursuing',

'studies', 'information', 'contribute', 'hidden', 'field', 'discover', 'tourist', 'communication', 'users', 'places', 'helps', 'across']
```